Universidad Nacional
Sede Región Brunca
EIF 212 Sistemas Operativos

Proyecto Programado

Sistema Concurrente de Gestión de Concierto Masivo

(Reservas y Venta de Entradas por Zonas)

1. Justificación Académica

La administración de recursos compartidos constituye uno de los problemas fundamentales en el diseño e implementación de Sistemas Operativos. La concurrencia, la sincronización de procesos y la prevención de interbloqueos son elementos esenciales para garantizar la integridad, consistencia y estabilidad de sistemas multiusuario.

El presente proyecto tiene como propósito modelar un entorno realista de acceso concurrente a recursos limitados, utilizando como escenario la gestión de un **concierto masivo dividido en múltiples zonas**. Este contexto permite simular conflictos de acceso, control de capacidad por zona, reservas simultáneas, transacciones con expiración y adquisición múltiple de recursos (por ejemplo, múltiples asientos contiguos), replicando problemas clásicos de concurrencia estudiados en el curso.

El lenguaje de programación es de libre escogencia por el estudiante (se recomienda C, C++, Java, C# o Python con manejo explícito de hilos).

2. Objetivo General

Diseñar e implementar un sistema concurrente cliente-servidor que gestione la reserva y venta de entradas para un concierto masivo, garantizando la integridad de los recursos compartidos mediante mecanismos explícitos de sincronización, exclusión mutua y prevención de interbloqueos.

3. Objetivos Específicos

- Modelar formalmente los recursos compartidos y secciones críticas del sistema.

- Implementar hilos o procesos concurrentes que simulen múltiples usuarios.
- Aplicar mecanismos de sincronización (mutex, semáforos, monitores).
- Prevenir condiciones de carrera y pérdida de actualización.
- Implementar estrategias formales para evitar interbloqueos.
- Validar el comportamiento del sistema bajo condiciones de alta concurrencia.
- Analizar formalmente las propiedades de correctitud (safety y liveness).

4. Descripción Formal del Sistema

El sistema simulará la gestión de venta de entradas para un concierto masivo dividido en múltiples zonas (por ejemplo: VIP, Preferencial, General).

Cada solicitud de reserva o compra será ejecutada como un hilo independiente que interactúa con los siguientes recursos compartidos críticos:

Recursos Compartidos

1. Matriz de Asientos por Zona

Estructura tridimensional:

Zona[i][fila][columna]

Donde:

- i identifica la zona del concierto.
- fila representa la fila dentro de la zona.
- columna representa el asiento dentro de la fila.

Estados posibles de cada asiento:

- Disponible
- Reservado

- Vendido

Cada zona posee capacidad limitada e independiente determinada por su número de filas y columnas.

La matriz de asientos constituye un **recurso compartido crítico**, ya que múltiples hilos pueden intentar modificar simultáneamente el estado de uno o más asientos.

2. Semáforo Contador por Zona

Cada zona deberá tener asociado un semáforo contador que represente la cantidad de asientos disponibles.

Este semáforo:

- Debe decrementar al reservar correctamente un asiento.
- Debe incrementarse cuando una reserva expire o sea cancelada.
- Debe permanecer consistente con el estado real de la matriz.

3. Tabla de Reservas Temporales

Estructura compartida que incluye:

- Identificador único de transacción.
- Zona asociada.
- Lista de asientos reservados (tuplas fila–columna).
- Timestamp de creación.
- TTL asociado.

Debe garantizarse coherencia entre esta tabla y la matriz de asientos.

4. Bitácora Global (Log Concurrente)

Registro de eventos concurrentes tales como:

- Solicitudes de reserva.
- Confirmaciones de compra.
- Cancelaciones.
- Expiraciones automáticas.

Debe ser tratada como recurso compartido y protegida contra accesos concurrentes inconsistentes.

5. Gestor de Transacciones

Componente encargado de:

- Confirmar compras.
- Cancelar reservas.
- Detectar y procesar expiraciones por TTL.
- Liberar asientos de manera segura.
- Mantener consistencia entre:
    - Matriz de asientos.
    - Semáforos por zona.
    - Tabla de reservas.

Arquitectura Cliente-Servidor

El sistema deberá seguir el modelo cliente-servidor:

Servidor

- Administra los recursos compartidos.
- Implementa mecanismos explícitos de sincronización.
- Atiende múltiples clientes concurrentes.

- Garantiza integridad y consistencia global del sistema.

Clientes

- Generan solicitudes concurrentes.
- Simulan usuarios adquiriendo entradas.
- Se conectan mediante sockets o mecanismo equivalente.

5. Requisitos Funcionales

El sistema deberá permitir:

- Consultar disponibilidad por zona.
- Reservar uno o varios asientos específicos (fila y columna) dentro de una zona.
- Confirmar compra.
- Cancelar reserva.
- Implementar reserva temporal con tiempo de expiración (TTL).
- Mostrar el estado global del sistema en tiempo real.
- Manejar múltiples zonas simultáneamente.

6. Requisitos No Funcionales

- Integridad absoluta de datos bajo concurrencia.
- Ausencia de condiciones de carrera.
- Prevención formal de interbloqueos.
- Liberación segura de locks.
- Estabilidad bajo carga concurrente alta (picos de demanda).
- Implementación explícita de mecanismos de sincronización (no se permite el uso de frameworks que abstraigan la concurrencia).

7. Modelado de Concurrencia

El estudiante deberá:

- Identificar claramente las secciones críticas sobre la matriz Zona[i][fila][columna].
- Definir un orden jerárquico global de adquisición de recursos (por ejemplo: zona → fila → columna) cuando se reserven múltiples asientos.
- Implementar exclusión mutua adecuada para la matriz de asientos.
- Controlar la capacidad mediante semáforos contadores por zona.
- Garantizar liberación segura de locks ante cancelaciones o expiraciones.
- Justificar formalmente la eliminación de al menos una condición de Coffman.
- Argumentar formalmente las propiedades de safety (no doble venta) y liveness (progreso del sistema).

8. Desarrollo del Proyecto por Fases

El proyecto se desarrollará en tres fases progresivas. Cada fase tendrá evaluación independiente y acumulativa.

8.1 Fase I – Análisis Formal y Diseño de Concurrencia (5%)

Descripción Detallada

El estudiante deberá realizar el modelado formal del sistema desde la perspectiva de Sistemas Operativos.

El documento deberá demostrar comprensión profunda de:

- Modelo de concurrencia seleccionado.
- Identificación explícita de recursos compartidos.
- Determinación formal de secciones críticas.
- Posibles condiciones de carrera.

- Escenarios potenciales de interbloqueo (especialmente en reservas múltiples).
- Estrategia formal de sincronización.
- Orden jerárquico de adquisición.
- Justificación técnica fundamentada.

Artefactos Esperados

- Documento técnico en PDF.
- Diagrama de arquitectura cliente-servidor.
- Diagrama de recursos compartidos.
- Tabla formal de secciones críticas.
- Modelo de sincronización adoptado.

Rúbrica Fase I (Evaluación Analítica)

Valor de la fase: 5%

Escala interna de evaluación: 0 – 100 puntos

Rango de Puntaje	Nivel de Desempeño	Descripción Integral del Modelado Formal
90 – 100 pts	Excelente	El modelado formal del sistema es completo, coherente y técnicamente fundamentado. Identifica con claridad los recursos compartidos, define correctamente las secciones críticas, analiza escenarios potenciales de condiciones de carrera e interbloqueo, establece un orden jerárquico de adquisición consistente y propone una estrategia de sincronización formalmente justificada. Utiliza terminología propia de Sistemas Operativos con precisión conceptual (exclusión mutua, safety, liveness, deadlock, etc.). No presenta inconsistencias estructurales.
75 – 89 pts	Muy Bueno	El modelado es sólido y mayormente coherente. Identifica adecuadamente los recursos y mecanismos de sincronización, aunque puede presentar omisiones menores, falta de profundidad en el análisis formal o especificaciones

Rango de Puntaje	Nivel de Desempeño	Descripción Integral del Modelado Formal
		parcialmente desarrolladas (por ejemplo, orden jerárquico no completamente formalizado o análisis de interbloqueo superficial). La comprensión conceptual es correcta pero no completamente rigurosa en todos los aspectos.
60 – 74 pts	Aceptable	El documento presenta una propuesta funcional pero con debilidades conceptuales visibles. Puede omitir recursos relevantes, definir secciones críticas de manera incompleta, no formalizar adecuadamente la estrategia de sincronización o analizar de manera limitada las condiciones de carrera e interbloqueo. Existen imprecisiones terminológicas o inconsistencias internas que afectan la claridad del modelo.
40 – 59 pts	Insuficiente	El modelado es incompleto o presenta inconsistencias técnicas importantes. Se identifican deficiencias significativas en la comprensión de concurrencia, sincronización o prevención de interbloqueo. La estrategia propuesta no está adecuadamente justificada o no corresponde con los riesgos identificados.
0 – 39 pts	Deficiente	No existe un modelado formal adecuado del sistema. Se evidencian errores conceptuales graves sobre concurrencia, exclusión mutua o correctitud. El documento no demuestra comprensión de los principios fundamentales del curso.

8.2 Fase II – Implementación Concurrente en Entorno Local (10%)

Descripción

Implementación en una sola máquina utilizando hilos reales.

Debe demostrar:

- Creación dinámica de hilos.
- Protección adecuada de recursos críticos.
- Control de capacidad mediante semáforos por zona.

- Implementación correcta de TTL.
- Registro concurrente seguro.
- Liberación adecuada de recursos.

Artefactos Esperados

- Código fuente completo comentado.
- Manual técnico breve.
- Evidencia de pruebas concurrentes (logs).

Rúbrica Fase II

Valor de la fase: 10%

Escala interna de evaluación: 0 – 100 puntos

Rango de Puntaje	Nivel de Desempeño	Descripción Integral de la Implementación Concurrente
90 – 100 pts	Excelente	La implementación utiliza hilos reales de forma explícita y correcta. Los recursos compartidos están adecuadamente protegidos mediante mecanismos de sincronización consistentes con el modelo formal. No se evidencian condiciones de carrera ni inconsistencias bajo pruebas repetidas de concurrencia. El control de capacidad por zona es coherente con el estado de la matriz de asientos. El TTL funciona correctamente y libera recursos de forma segura. La estructura del sistema es modular, clara y técnicamente sólida.
75 – 89 pts	Muy Bueno	La implementación es funcional y mayormente estable bajo concurrencia. Puede presentar imprecisiones menores en sincronización, control de capacidad o manejo del TTL, sin comprometer la integridad general del sistema. Existen pequeños riesgos potenciales o debilidades estructurales que no generan fallos críticos reproducibles. El diseño es comprensible aunque no completamente robusto.

Rango de Puntaje	Nivel de Desempeño	Descripción Integral de la Implementación Concurrente
60 – 74 pts	Aceptable	El sistema funciona en escenarios básicos pero presenta debilidades evidentes bajo carga concurrente. Pueden observarse comportamientos inconsistentes no críticos, sincronización parcialmente correcta o manejo incompleto de recursos. La implementación de hilos es válida pero no completamente rigurosa. Existen vulnerabilidades técnicas que afectan la robustez del sistema.
40 – 59 pts	Insuficiente	La implementación presenta inconsistencias significativas bajo concurrencia. Se detectan problemas en la protección de recursos críticos, manejo inadecuado de sincronización o liberación incorrecta de recursos. Pueden existir fallos reproducibles que comprometen parcialmente la propiedad de safety. La estructura del código dificulta su análisis técnico.
0 – 39 pts	Deficiente	No existe una implementación concurrente adecuada. Se detectan condiciones de carrera reproducibles, doble reserva confirmada, corrupción de datos o ausencia de mecanismos explícitos de sincronización. El sistema no demuestra comprensión práctica de los principios de concurrencia estudiados.

8.3 Fase III – Implementación Distribuida y Validación Bajo Carga (10%)

Descripción

Despliegue en servidor remoto y pruebas con múltiples clientes reales.

Debe demostrar:

- Comunicación estable cliente-servidor.
- Sincronización correcta bajo acceso remoto.
- Estabilidad ante pruebas de estrés.

- Integridad del sistema bajo múltiples conexiones.

Artefactos Esperados

- Sistema desplegado.
- Instrucciones de conexión.
- Registro de pruebas de carga.
- Presentación técnica para defensa.

8.3.1 Generador Obligatorio de Carga Concurrente

Con el fin de validar formalmente el comportamiento del sistema bajo condiciones reales de alta concurrencia, el estudiante deberá implementar un módulo adicional denominado **Generador de Carga Concurrente**.

Este módulo deberá:

- Generar al menos 100 solicitudes concurrentes reales hacia el servidor desplegado.
- Permitir parametrización del número de solicitudes (100, 200, 500 o más).
- Ejecutar solicitudes de manera simultánea (no secuencial).
- Incluir escenarios conflictivos (intentos simultáneos sobre los mismos asientos).
- Registrar resultados con timestamps y estado final de cada transacción.

Durante la defensa técnica, el grupo y la docente podrán ejecutar dicho generador en tiempo real contra el servidor del estudiante con el fin de observar:

- Comportamiento bajo carga.
- Integridad de la matriz de asientos.
- Coherencia de semáforos por zona.
- Ausencia de doble venta confirmada.

La ausencia de este generador limitará la calificación máxima posible en la Fase III.

Universidad Nacional
Sede Región Brunca
EIF 212 Sistemas Operativos

Rúbrica Fase III

Valor de la fase: 10%

Escala interna de evaluación: 0 – 100 puntos

Rango de Puntaje	Nivel de Desempeño	Descripción Integral de la Implementación Distribuida
90 – 100 pts	**Excelente**	El sistema distribuido es estable, consistente y técnicamente sólido bajo múltiples conexiones concurrentes. La arquitectura cliente–servidor está claramente implementada y documentada. No se observan violaciones de safety bajo pruebas de estrés repetidas. La sincronización es coherente con el modelo formal diseñado en Fase I. El sistema mantiene liveness incluso bajo alta carga. Se presentan métricas cuantitativas y análisis crítico del comportamiento. La defensa técnica demuestra dominio conceptual profundo.
75 – 89 pts	**Muy Bueno**	El sistema funciona correctamente en entorno distribuido y soporta concurrencia real. Puede presentar limitaciones menores bajo carga extrema (por ejemplo, degradación de rendimiento o pequeñas ineficiencias), pero no compromete la integridad del sistema. La validación experimental es adecuada aunque no completamente exhaustiva. La arquitectura es correcta pero podría optimizarse.
60 – 74 pts	**Aceptable**	El sistema opera de forma distribuida pero presenta debilidades visibles bajo carga elevada. Pueden observarse bloqueos temporales prolongados, degradación significativa de rendimiento o manejo incompleto de desconexiones. No se evidencian violaciones críticas de safety, pero la robustez general es limitada. El análisis técnico es superficial o incompleto.
40 – 59 pts	**Insuficiente**	La implementación distribuida presenta fallos importantes bajo concurrencia remota. Se detectan inconsistencias, bloqueos frecuentes o errores de sincronización. Las pruebas de carga son insuficientes o mal documentadas. La arquitectura

Rango de Puntaje	Nivel de Desempeño	Descripción Integral de la Implementación Distribuida
		no está claramente definida o no se corresponde con el modelo formal propuesto.
0 – 39 pts	**Deficiente**	El sistema no funciona correctamente en entorno distribuido o presenta violaciones reproducibles de safety (doble venta confirmada, corrupción de datos). No existe validación bajo carga o la arquitectura cliente–servidor no está correctamente implementada. Se evidencian errores conceptuales graves sobre concurrencia distribuida.

Cláusula Formal de Correctitud

La presencia de una violación reproducible de la propiedad de safety (por ejemplo, doble venta confirmada de un mismo asiento (zona, fila, columna)) constituye error crítico de concurrencia y podrá afectar la aprobación del proyecto, independientemente del cumplimiento de otros criterios.

Nota Importante

Se prohíbe el uso de bibliotecas de alto nivel que abstraigan la implementación de concurrencia.

El estudiante deberá demostrar control explícito de:

- Hilos.
- Exclusión mutua.
- Semáforos.
- Manejo de recursos.
- Razonamiento formal sobre correctitud.

Notas Generales

- El proyecto se realizará en parejas (dos estudiantes por equipo).

- El proyecto **no podrá desarrollarse como aplicación web**. Deberá implementarse como aplicación de escritorio.